

Nhật ký sử dụng công cụ AI

Công cụ: Claude Code (Anthropic), mô hình Claude Sonnet 5, chạy trong VSCode. **Cách dùng:** AI được dùng như một lập trình viên cặp đôi (pair-programmer) trực tiếp viết code/tài liệu theo yêu cầu, con người (tôi) đọc, kiểm tra, chạy thử, và **chủ động sửa hướng đi của AI** khi phát hiện đi lệch — không copy-paste mù quáng. Các mốc sửa hướng quan trọng được ghi rõ ở mục 2, 4, 8 và 11 bên dưới.

1. Giai đoạn 1 — Xây dựng khung dự án ban đầu

Yêu cầu đưa cho AI: xây web quản lý thuê nhà bằng smart contract, tách rõ UI và BE để có thể thiết kế giao diện độc lập bằng dữ liệu giả trước khi có contract thật.

AI thực hiện: đề xuất và code kiến trúc service layer (`RentalService` là interface chung, `MockRentalService` dùng dữ liệu giả, `ChainRentalService` gọi contract thật qua ethers.js) để component UI không biết mình đang chạy mock hay chain thật; viết smart contract `RentalManager.sol` (khi đó là một contract duy nhất) với đầy đủ 8 chức năng tối thiểu; viết test Hardhat/Chai; viết tài liệu phân tích bài toán, kiến trúc, thiết kế dữ liệu.

Con người kiểm tra: đọc lại kiến trúc tách UI/BE, xác nhận hợp lý trước khi cho AI tiếp tục sang các phần khác.

2. Giai đoạn 2 — Hỗ trợ deploy Vercel & Sepolia testnet

Yêu cầu: hướng dẫn deploy frontend lên Vercel với auto-deploy khi commit; hướng dẫn tạo ví MetaMask, lấy RPC URL (Alchemy), lấy SepETH từ faucet, deploy thật lên Sepolia.

AI thực hiện: hướng dẫn từng bước qua các màn hình cụ thể (Alchemy, MetaMask, faucet) dựa trên ảnh chụp màn hình người dùng gửi.

Điểm can thiệp bảo mật quan trọng: trong lúc thao tác, người dùng vô tình dán **private key** vào ô "Wallet Address" của một faucet. AI phát hiện, cảnh báo rõ ràng (⚠️), giải thích khác biệt giữa private key và địa chỉ ví công khai, và khuyến nghị tạo ví mới thay vì tiếp tục dùng ví có khả năng đã lộ khoá — đây là một ví dụ AI **chủ động ngăn một hành động có rủi ro bảo mật thật**, không chỉ làm theo yêu cầu.

3. Giai đoạn 3 — Rà soát lại theo quy trình chuẩn (mốc sửa hướng lớn nhất)

Bối cảnh: được cung cấp 6 tài liệu mô tả quy trình chuẩn xây dựng ứng dụng blockchain dựa trên token (minh hoạ bằng ví dụ "quản lý chứng chỉ NFT"). Yêu cầu: rà soát xem đề tài thuê nhà đã đi đúng quy trình 0–5 hay chưa.

Sai sót của AI cần được sửa: AI hiểu nhầm ban đầu và **bắt đầu xây một dự án mới, tách biệt** để "quản lý chứng chỉ NFT" — đúng theo ví dụ minh hoạ trong tài liệu, nhưng sai với ý định thật của người dùng (ví dụ minh hoạ không phải đề tài phải làm).

Sửa hướng bởi con người: người dùng chỉ rõ — 6 tài liệu là **quy trình chung**, ví dụ chứng chỉ NFT chỉ minh hoạ cách áp dụng quy trình, **không phải đề tài cần làm**; yêu cầu thật là *áp dụng quy trình đó vào đúng đề tài thuê nhà đang có*, không tạo dự án song song. Đây là minh chứng rõ nhất trong toàn bộ quá trình về việc con người phải đọc kỹ và chặn lại khi AI đi sai hướng, thay vì để AI tự quyết định phạm vi công việc.

AI điều chỉnh sau khi được sửa: rà soát lại `RentalManager.sol` so với quy trình, xác định lịch lớn nhất là chưa dùng chuẩn token nào (không ERC-20/721/1155) và chưa tách token contract khỏi business contract — lập kế hoạch 5 phase để khắc phục (trình bày cho người dùng duyệt trước khi code, qua chế độ Plan Mode).

4. Giai đoạn 4 — Quyết định phạm vi công việc (do con người chọn)

AI đặt câu hỏi rõ ràng để con người quyết định phạm vi thay vì tự đoán:

- Có làm luôn Bước 4 (backend indexer + Slither + checklist production) trong lần này không, hay để sau? → **Người dùng chọn: làm luôn.**
- Giữ Hardhat 2/JavaScript hay chuyển hẳn sang Hardhat 3 + TypeScript để bám sát 100% công cụ trong tài liệu mẫu? → **Người dùng chọn: chuyển sang Hardhat 3 + TypeScript.**

5. Giai đoạn 5 — Thực thi kế hoạch 5 phase

AI thực hiện (đã được duyệt kế hoạch trước):

- Migrate toolchain Hardhat 2/JS → Hardhat 3/TypeScript trước, xác minh 11 test cũ vẫn pass trên toolchain mới trước khi đổi kiến trúc contract (tách rủi ro đổi công cụ ra khỏi rủi ro đổi kiến trúc).
- Tách `RentalAgreementToken.sol` (token) khỏi `RentalManager.sol` (ng nghiệp vụ), giữ nguyên chữ ký hàm/event công khai của `RentalManager` để frontend không cần sửa.
- Viết tài liệu Bước 1 (lựa chọn loại token, so sánh ERC-20/721/1155).
- Xây backend event indexer + REST API.
- Cài Slither, bổ sung test bảo mật, viết checklist production.

Lỗi kỹ thuật AI tự phát hiện và tự sửa trong lúc thực thi (không cần con người chỉ ra) — liệt kê để cho thấy AI có tự kiểm tra/tự sửa, không chỉ sinh code một lần rồi dừng:

Lỗi gặp phải	Cách AI phát hiện	Cách sửa
<code>changeEtherBalance</code> báo lỗi kiểu dữ liệu sau khi lên Hardhat 3	Đọc source code <code>node_modules</code> của <code>matcher</code> để tìm chữ ký hàm mới	Thêm tham số <code>ethers</code> đầu tiên theo API mới
<code>mcopy</code> opcode not found khi build	Đọc thông báo lỗi, xác định đây là opcode của hardfork Cancun	Thêm <code>evmVersion: "cancun"</code> vào cấu hình solidity
Lỗi <code>mcopy</code> vẫn xảy ra riêng khi deploy bằng Ignition dù build/test đã qua	Đọc source code Hardhat để tìm ra hệ thống "build profiles" tách biệt	Thêm cờ <code>--build-profile default</code> vào lệnh deploy
Cài <code>better-sqlite3</code> thất bại (thiếu Visual Studio Build Tools trên máy)	Đọc log lỗi <code>node-gyp</code>	Đổi sang dùng <code>node:sqlite</code> (module có sẵn trong Node.js, không cần biên dịch native)
Slither không phân tích được project Hardhat 3	Đọc traceback lỗi <code>crytic-compile</code> , thử nhiều đường vòng, cô lập bằng cách gọi thẳng <code>solc</code> để xác nhận lỗi nằm ở <code>solc</code> / <code>crytic-compile</code> , không phải ở contract	Không có cách khắc phục triệt để trong phạm vi thời gian cho phép — AI ghi nhận trung thực giới hạn này trong <code>bao-mat-slither.md</code> thay vì giả vờ đã chạy xong, và làm rà soát bảo mật thủ công thay thế

Nguyên tắc được giữ xuyên suốt: **không đánh dấu ✓ giả** cho hạng mục chưa thực sự làm được (vd Slither, Foundry fuzz test, audit độc lập) — ghi rõ trong `production-checklist.md` đây là hạng mục còn thiếu.

6. Giai đoạn 6 — Demo end-to-end

Yêu cầu: chạy demo thực tế.

AI thực hiện: tự khởi động Hardhat local node, deploy contract, tạo giao dịch mẫu thật trên chain, khởi động backend (indexer + REST API), khởi động frontend, dùng trình duyệt headless (Playwright) để chụp màn hình và kiểm tra không có lỗi console — chứng minh toàn bộ hệ thống hoạt động cùng nhau thật sự, không chỉ code biên dịch được.

7. Giai đoạn 7 — Cập nhật tài liệu & hoàn thiện sản phẩm nộp bài

Yêu cầu: cập nhật lại tài liệu phân tích bài toán cho khớp với những gì đã build (vì tài liệu gốc viết trước khi tách token/backend); liệt kê lại toàn bộ cấu trúc thư mục và đối chiếu với danh sách sản phẩm cần nộp; hoàn thành các tài liệu còn thiếu (báo cáo kết quả, phân công công việc, nhật ký sử dụng AI này).

AI thực hiện: cập nhật `phan-tich-bai-toan.md`, liệt kê cây thư mục đầy đủ và đối chiếu với yêu cầu đề bài, viết `bao-cai-cao-ket-qua.md`, `phan-cong-cong-viec.md`, và tài liệu này.

8. Giai đoạn 8 — Triển khai thật 3 chức năng nâng cao (trọng tài, phạt trễ, multisig)

Yêu cầu: "làm nốt phần nâng cao" — 3 chức năng khi đó còn ghi "chưa code": cơ chế trọng tài khi tranh chấp, phạt thanh toán trễ, multisig cho giải ngân tiền cọc.

AI thực hiện: thiết kế lại `RentalManager.sol` — thêm `AccessControl` (chỉ cho `ARBITER_ROLE`, nghiệp vụ chính vẫn phi tập trung), enum `Status` thêm `Disputed`, luồng `proposeSettlement` → `acceptSettlement/disputeSettlement` → `voteOnDispute` thay cho `endLease` cũ, đếm phiếu trọng tài trực tiếp on-chain thay vì dùng `TimelockController` riêng (đơn giản hơn, đánh đổi là không có độ trễ thời gian — ghi rõ trong `gioi-han-va-rui-ro.md`). Viết lại 28 test, deploy lại Sepolia, đồng bộ frontend.

Điểm can thiệp bảo mật quan trọng thứ hai: AI phát hiện địa chỉ người dùng định cấp `ARBITER_ROLE` trùng với **private key thử nghiệm công khai của Hardhat** (ai cũng biết khoá này). AI chủ động cảnh báo rủi ro cụ thể cho mạng công khai (Sepolia): người lạ cũng có thể tự ký giao dịch bằng đúng khoá đó. Người dùng cân nhắc và **chọn chấp nhận rủi ro** vì chỉ là đồ án demo — AI tôn trọng quyết định, không tự ý chặn, nhưng đã đưa đủ thông tin để con người quyết định có hiểu biết (informed decision), không phải mù quáng làm theo.

9. Giai đoạn 9 — Gỡ lỗi hệ thống thật trên Sepolia (chẩn đoán có hệ thống, không đoán mò)

Sau khi deploy thật, phát sinh nhiều lỗi chỉ xuất hiện trên mạng thật (không tái hiện được lúc dev local) — AI xử lý bằng cách **tự tra cứu trực tiếp on-chain qua RPC công khai** (độc lập với giao diện) để tách bạch lỗi do hợp đồng/dữ liệu hay do code frontend, trước khi sửa bất cứ gì:

Lỗi	Cách chẩn đoán (không đoán mò)	Nguyên nhân thật	Cách sửa
Nhập giá dùng dấu phẩy ("0,01") bị từ chối	Đọc code <code>ethers.parseEther</code>	Chỉ chấp nhận dấu <code>.</code>	Hàm <code>parseEth()</code> tự đổi <code>,</code> → <code>.</code> trước khi parse
Đăng tin thành công (MetaMask báo "Đã xác nhận") nhưng web hiện "0 phòng"	Gọi thẳng <code>eth_call / getAllProperties()</code> qua RPC công khai (<code>publicnode.com</code>), so với dữ liệu web hiển thị	<code>loadHistory()</code> quét log từ block 0, vượt giới hạn số block mỗi lần gọi <code>eth_getLogs</code> của RPC (đã gập cả mức 50000 lần 10000 tùy node) → lỗi này làm cả <code>Promise.all reject</code> , xoá mất luôn dữ liệu <code>loadProperties()</code> vốn đã tải thành công	Quét log theo từng đoạn nhỏ (<code>queryFilterChunked</code>) tính từ block deploy; tách riêng <code>try/catch</code> cho <code>loadProperties()</code> và <code>loadHistory()</code> để một lỗi không xoá mất kết quả của lỗi kia
Ảnh phòng không hiện	Tải hiện bằng Playwright + provider giả lập trò RPC thật, đọc console log	Gateway <code>ipfs.io</code> không phản hồi (dù ảnh đã upload Pinata thành công)	Đổi gateway đọc ảnh sang <code>gateway.pinata.cloud</code> (cùng nơi ảnh được lưu)
<code>npm run deploy:sepolia</code> báo "Nothing new to deploy", vẫn giữ địa chỉ contract cũ dù đã sửa code	So sánh độ dài bytecode đã biên dịch mới (artifact) với bytecode thật trên chain (<code>eth_getCode</code>) — phát hiện lệch, xác nhận contract chưa hề được deploy lại	Hardhat Ignition so khớp theo tham số constructor (không đổi), không phát hiện mã nguồn bên trong đã đổi, nên bỏ qua deploy	Xoá thư mục <code>ignition/deployments/chain-11155111/</code> (đã commit, khôi phục được) để ép deploy lại từ đầu
Sau khi thao tác xong, trạng thái không tự cập nhật, phải F5	Viết kịch bản Playwright thao tác trên chain local (mine tức thì) — xác nhận cơ chế tự tải lại vốn hoạt động đúng khi không có độ trễ mạng	RPC thật của Sepolia (thường là cụm nhiều node sau load balancer) đôi lúc đọc phải node chưa kịp đồng bộ ngay sau khi ghi	Tự đọc lại lần 2 sau 1.5 giây cho mọi giao dịch trên chain thật (không cần với mock, luôn tức thời)

Tất cả các chẩn đoán trên đều dựa vào **bằng chứng lấy trực tiếp từ RPC/on-chain** (không phải suy đoán từ triệu chứng), viết bằng script Node độc lập trước khi sửa code — tránh sửa sai chỗ.

10. Giai đoạn 10 — Chức năng bổ sung `cancelListing` (do AI đề xuất, người dùng duyệt)

Người dùng đặt câu hỏi mở: "vì dữ liệu blockchain nếu lỡ đăng nhầm giá thì sao nhỉ" — AI xác nhận đúng là không sửa được, kiểm chứng bằng cách đọc lại `rentProperty` (`require msg.sender != p.landlord` — chủ nhà không tự thuê để "dọn" tin sai được), rồi **chủ động đề xuất giải pháp** (`cancelListing`) kèm đánh đổi (cần deploy lại) thay vì chỉ trả lời "không có cách nào". Người dùng đồng ý, AI code, viết 4 test, kiểm thử end-to-end thật trên chain local bằng Playwright + tài khoản Hardhat trước khi deploy Sepolia (đăng tin giá sai → bấm huỷ → xác nhận trạng thái đổi đúng, không cần F5).

Sau khi deploy lại (dữ liệu cũ "biến mất" khỏi web vì contract mới rỗng), người dùng hỏi có cách khôi phục không — AI đọc lại toàn bộ 15 tin từ contract cũ qua RPC, phân loại rõ 3 nhóm (12 tin đăng lại tự động được, 2 tin thuộc ví khác

không đăng thay được, 1 tin đã có người thuê/lich sử không tái tạo được), viết `scripts/relist-from-old-contract.ts` chỉ đăng lại đúng nhóm khôi phục được, không giả vờ khôi phục được toàn bộ.

11. Giai đoạn 11 — Lỗi xung đột lợi ích khi bỏ phiếu trọng tài (do người dùng phát hiện)

Người dùng phát hiện: khi đứng vai chủ nhà (ví admin) vẫn thấy được nút "Bỏ phiếu trọng tài" cho tranh chấp của chính mình — nhận ra ngay đây là bất hợp lý dù AI chưa chỉ ra trước.

AI xác nhận nguyên nhân: đọc lại constructor `RentalManager.sol`, `_grantRole(ARBITER_ROLE, admin)` — admin (ví deploy, cũng là ví chủ nhà lúc test) mặc định là trọng tài #1. Đề xuất 2 hướng sửa: (1) sửa contract chặn hẳn chủ nhà/người thuê bỏ phiếu cho tranh chấp của mình (cần deploy lại), (2) thu hồi quyền trọng tài khỏi ví admin, cấp cho ví độc lập thứ 3 (không cần deploy lại). Người dùng chỉ chọn hướng (2) — AI viết `scripts/revoke-arbiter.ts`, thực hiện, xác minh lại bằng `hasRole()` qua RPC, và **ghi nhận trung thực** trong `gioi-han-va-rui-ro.md` rằng hướng (1) — chặn ở mức contract — vẫn chưa làm, đây là giới hạn còn tồn tại chứ không phải đã khắc phục triệt để.

Lỗi của AI trong lúc hỗ trợ debug bước này: khi người dùng test bỏ phiếu bằng 2 ví trọng tài và báo lỗi "đã bỏ phiếu rồi" dù chưa vote, AI **chẩn đoán sai lần đầu** — nghi ngờ nhầm là lỗi MetaMask không đồng bộ quyền kết nối theo từng site. Chỉ khi người dùng gửi ảnh chụp rõ tên tài khoản MetaMask ("đây mới đúng là trọng tài 2"), AI mới nhận ra mình đã **gán nhầm nhãn "Trọng Tài 1"/"Trọng Tài 2" với sai địa chỉ** trong lúc trò chuyện trước đó — lỗi thật ra không phải kỹ thuật (contract/frontend vẫn đúng), mà do AI tự nhầm lẫn tên gọi. Xin lỗi và sửa lại ngay khi phát hiện, không che giấu.

12. Đánh giá mức độ can thiệp của con người

- AI **không** được để tự quyết định phạm vi hay kiến trúc quan trọng mà không hỏi — hai lần dùng câu hỏi trực tiếp (Giai đoạn 4) để con người chọn thay vì AI tự đoán.
- Có **một lần AI hiểu sai nhiệm vụ** (Giai đoạn 3) và bị con người sửa lại bằng phản hồi trực tiếp, rõ ràng — đây là bằng chứng cho thấy kết quả cuối cùng có sự giám sát và chỉnh sửa của con người, không phải chấp nhận nguyên si output đầu tiên của AI. Toàn bộ nội dung tin nhắn sửa hướng gốc được lưu trong lịch sử làm việc, không chỉnh sửa lại ở đây.
- Mọi lỗi kỹ thuật phát sinh trong lúc code (bảng ở mục 5) đều được AI tự phát hiện qua đọc log lỗi/đọc source code thư viện, không đoán mò; con người xác nhận bằng cách chạy lại test (`npx hardhat test`) sau mỗi lần sửa để chắc chắn không hồi quy.
- Giới hạn thật của công cụ (Slither không chạy được) được ghi nhận trung thực thay vì che giấu — coi đây là nguyên tắc bắt buộc xuyên suốt dự án.
- Sau khi deploy thật (Giai đoạn 8–11), phần lớn lỗi phát sinh chỉ xuất hiện trên mạng thật — AI luôn **kiểm chứng bằng RPC/on-chain trực tiếp trước khi sửa code**, không sửa theo phỏng đoán (bảng ở mục 9). Có **hai lần AI mắc lỗi thật sự trong giai đoạn này** (chẩn đoán sai nguyên nhân báo lỗi "đã bỏ phiếu rồi" ở mục 11; và trước đó là các mốc đã ghi ở mục 3) — cả hai đều được ghi nhận công khai, không chỉnh sửa lại lịch sử để trông "sạch" hơn thực tế.
- Một lỗi thiết kế bảo mật (xung đột lợi ích khi bỏ phiếu trọng tài, mục 11) do chính người dùng phát hiện ra trước AI** — AI xác nhận nguyên nhân, đề xuất 2 hướng sửa kèm đánh đổi rõ ràng, để người dùng chọn mức độ khắc phục (chỉ sửa vận hành, chưa sửa contract) thay vì AI tự quyết định mức độ "đủ an toàn".

